

altairsim — Recipe: starting from a machine that already works

2026-09-23 · b1a286b

Building a machine board by board is worth doing once. After that, start from one that already runs and change what you do not like — which is what everybody actually does, and what the shipped machine files do.

Same four beats as every recipe here, with one different first move:

```
altairsim -n, then MACHINE default — a whole working Altair, in one line. Change it — BOARDS  
REMOVE, SET, MOUNT, STARTUP ADD. CONFIG SAVE — write it down. Quit, start again, CONFIG  
LOAD — and it comes back.
```

Work in a copy of `examples/cpm/`, which has the disk this recipe boots.

```
$ cd examples/cpm
```

1. See what is on the shelf

Machines are built into altairsim. `SHOW MACHINES` lists them, each with a line saying what it is:

```
$ altairsim -n  
altairsim> SHOW MACHINES  
NAME          DESCRIPTION  
-----  
8085           A minimal 8085 machine: an `8085` CPU, 64K of RAM, and a  
                2SIO console.  
basic4k        The machine Altair 4K BASIC was sold to run on: an 88-SIO  
                Teletype, a cassette in the ACR.  
dazzler        A Cromemco Dazzler in an Altair -- the bench for the  
                S-100's first color graphics card.  
default        The machine you get when you name none: 56K, and the DBL  
                boot PROM at FF00.
```

— and more; the list goes on past what is printed here.

SHOW MACHINE <name> opens one up without loading it, so you can see exactly what is in a machine before you decide to take it:

```
altairsim> SHOW MACHINE default
name      default
          The machine you get when you name none: 56K, and the DBL boot PROM
          at FF00.
startup   (none)

  ID      TYPE      I/O      UNITS      MEMORY
  ----      -
fp0  fp      FF      -      -
cpu0 8080      -      1 cpu: 8080      -
sio0 2sio     10,12    2 serial: a*, b  -
dsk0 dcdd     08,09,0A  4 disk: drive0(empty), drive1(empty), drive2(empty), drive3(empty) -
hb0  hostbridge B0,B1    -      -
mem0 memory    -      1 rom: rom0      0000-DFFF
ram  56K
rom  dbl phantom:all      FF00-FFFF

* holds the console
```

That is the whole 56K CP/M Altair: front panel, 8080, serial board with your terminal on it, floppy controller with four empty drives, the host bridge, 56K of RAM and the boot loader in ROM at FF00. A bare **SHOW MACHINE**, with no name, shows the machine you are running instead.

2. Take it

```
altairsim> MACHINE default
machine default: 6 board(s)
```

One line, and every board you just looked at is in the backplane. **MACHINE <name>** replaces whatever was there, and it is the command form of a machine file's `base = "<name>"`.

It gives you the **hardware** and not the boot: a built-in's own **startup** is not run, which is why the drives are empty above and why you do the **MOUNT** and the **RUN** yourself below.

3. Change it

Three edits: pull a board out, slow the processor down to the real thing, put a disk in.

```
altairsim> BOARDS REMOVE hb0
hb0: removed
altairsim> SET cpu0 clock_hz=2000000
cpu0: clock_hz=2000000
altairsim> MOUNT dsk0:drive0 cpm22b23-56k.dsk
dsk0:drive0: mounted cpm22b23-56k.dsk
altairsim> STARTUP ADD RUN FF00
added: RUN FF00
altairsim> SET MACHINE name=altair2mhz
machine: name=altair2mhz
```

BOARDS REMOVE is the other half of **BOARDS ADD**. The host bridge is a modern convenience — it moves files between the guest and your own disk — and a machine meant to be period-correct has no such thing. Out it comes.

clock_hz belongs to the processor board, because the crystal was on that card. **0** is the default and means flat out; **2000000** is the 88-CPU's real 2 MHz, with the real waiting that comes with it. **SHOW cpu0** prints what a board holds and what it will take:

```
altairsim> SHOW cpu0
cpu0 (8080)

  unit      kind    holds
   8080     cpu     active

property      value      legal
clock_hz      2000000    0..100000000
idle          true      true|false
achieved_hz   0         (read-only)
```

SET takes one property per command — **SET cpu0 clock_hz=2000000 idle=off** is two commands, not one.

SET MACHINE name= is the machine's own name, not a board's. It came off the shelf called **default**, and this is no longer the default machine, so it gets a name of its own — the one **CONFIG SAVE** writes into the file.

4. Save it, quit, load it back

SHOW MACHINE is what you are about to save — the name, the startup list, and the backplane as it now stands:

```
altairsim> SHOW MACHINE
name      altair2mhz
startup
    RUN FF00

  ID  TYPE  I/O  UNITS  MEMORY
  ---  ---  ---  ---
fp0   fp    FF    -      -
cpu0  8080    -    1 cpu: 8080  -
sio0  2sio  10,12  2 serial: a*, b  -
dsk0  dcdd   08,09,0A  4 disk: drive0, drive1(empty), drive2(empty), drive3(empty) -
mem0  memory -      1 rom: rom0  0000-DFFF ram 56K
                                FF00-FFFF rom dbl

phantom:all

* holds the console
```

Five boards, the host bridge gone, and a name of its own.

```
altairsim> CONFIG SAVE altair2mhz.toml
saved altair2mhz.toml
altairsim> QUIT
```

```
$ altairsim -n
altairsim> CONFIG LOAD altair2mhz.toml
loaded altair2mhz.toml: 5 board(s)
startup> RUN FF00
[console -- ^E returns to the monitor]

56K CP/M 2.2b v2.3
For Altair 8" Floppy

A>
```

Five boards, not six — the one you pulled stayed pulled. Ctrl-E gets you back to the **altairsim>** prompt, and `$ altairsim altair2mhz.toml` does the same thing as the two lines above.

5. The short way to write the same file

`CONFIG SAVE` writes down the whole machine: every board, every setting, including everything you never touched. That is right for a file you want to keep working, and it is more than you need to *read*.

A machine file can instead say "the default machine, with these differences":

```
[machine]
name      = "altair2mhz"
base      = "default"
startup   = ["RUN FF00"]

[[board]]
id        = "hb0"           # the host bridge, pulled back out
remove    = true

[[board]]
id        = "cpu0"          # no `type`: change the board that is already there
clock_hz  = 2000000

[[board]]
id        = "dsk0"

[[board.drive]]
unit      = 0
mount     = "cpm22b23-56k.dsk"
```

`base` is the same thing as the `MACHINE default` you typed in step 2, and the rest is your three edits, one block each. That is the trade: a `base` file is short and says what you meant, a saved file is long and complete.

A `[[board]]` with an `id` and no `type` changes the board the base already gave you. Write `type = "8080"` there as well and you throw the base's processor away and fit a fresh one with default settings — which still works, and quietly undoes your `clock_hz`. Leave the `type` out.

This is why the shipped machine files are a few lines long. Open `examples/cpm/cpm22-buffered.toml` and you will recognise every part of it.